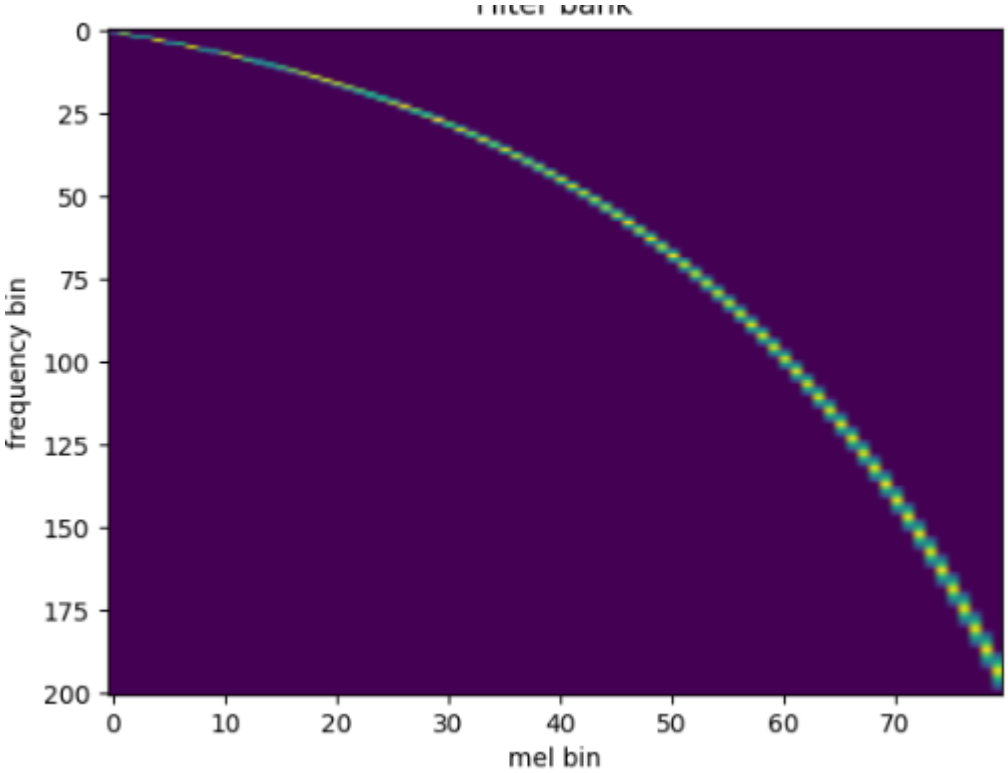
Результаты:

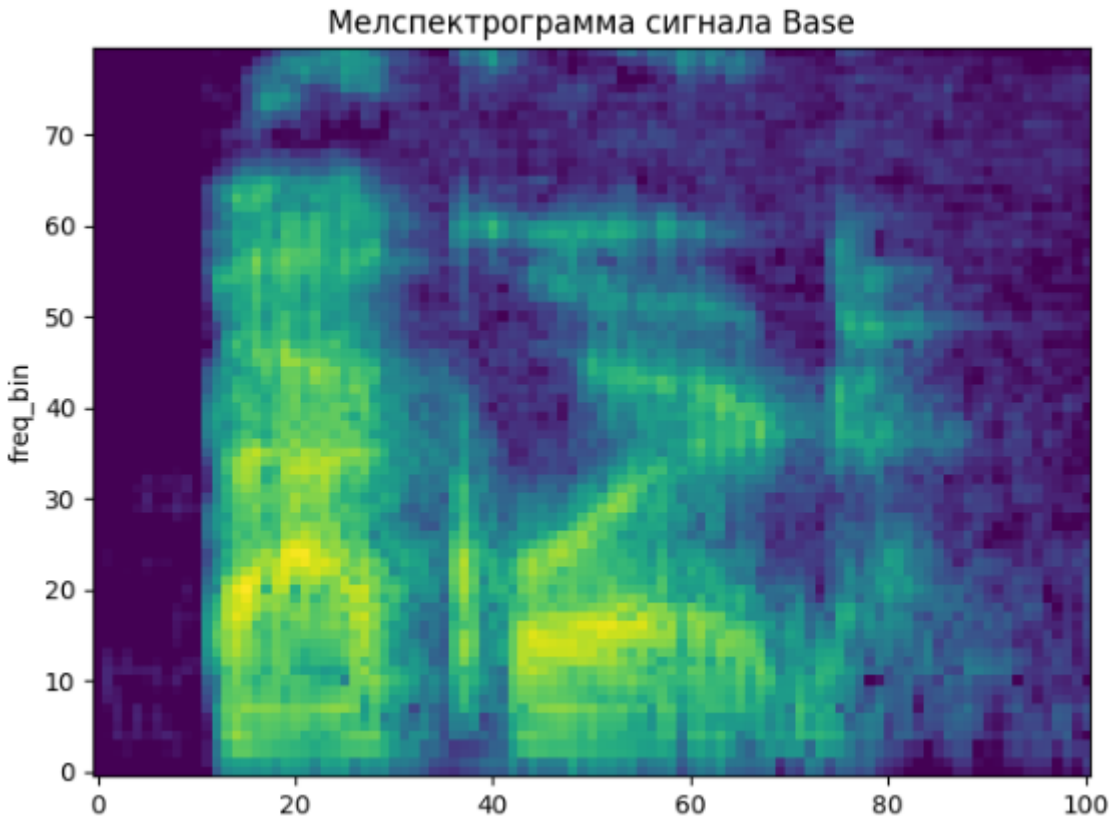
1. Реализация кастомного LogMelFilterBanks

Реализован класс `LogMelFilterBanks(nn.Module)`, который последовательно считает STFT с окном Ханна, берёт степенной спектр, умножает на мелфильтр банки и применяет логарифм.

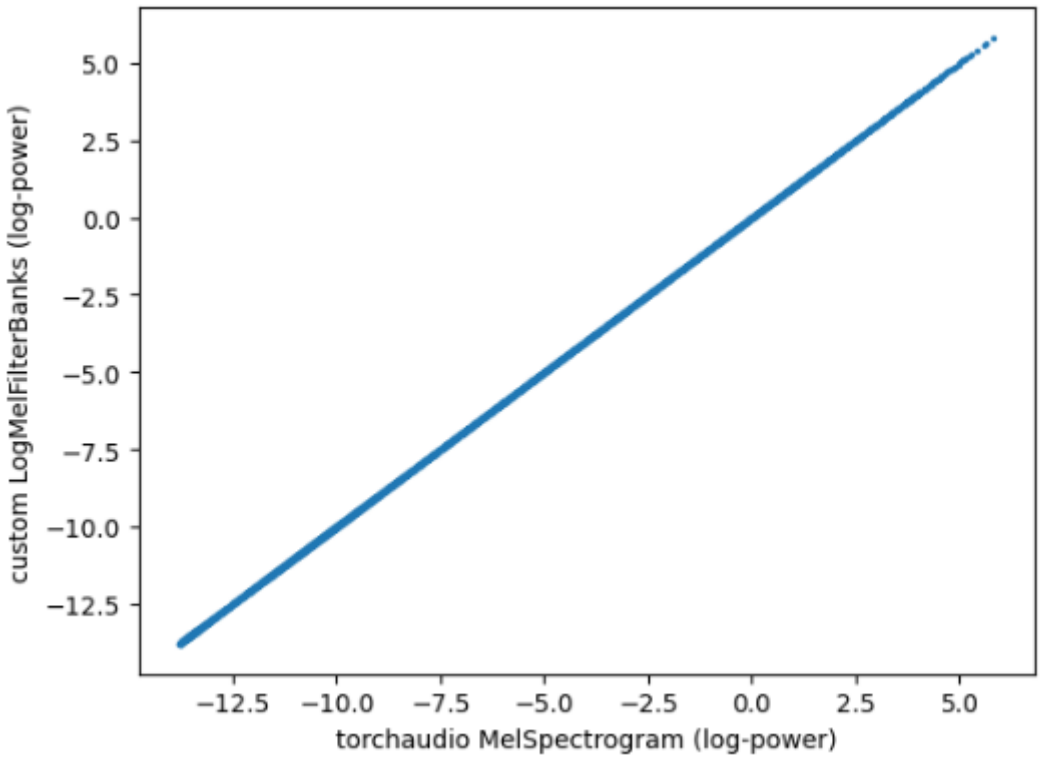
Мелфильтр банки (80 фильтров):



Мелспектрограмма сигнала (без log):



Сравнение LogMelFilterBanks vs torchaudio + log:



Выход кастомной реализации совпадает с `torchaudio` + log - точки лежат строго на диагонали, allclose проходит.

2. Обучение модели классификации

Реализована модель MelNet на Conv1d свёртках:

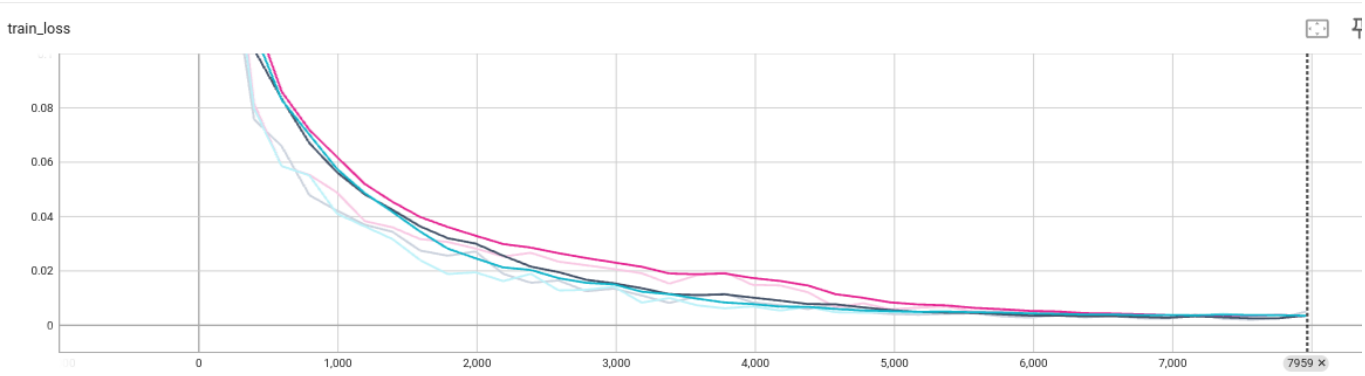
```
        Layer (type)               Output Shape         Param #
================================================================
            Conv1d-1              [-1, 64, 101]          25,664
       BatchNorm1d-2              [-1, 64, 101]             128
         MaxPool1d-3               [-1, 64, 50]               0
            Conv1d-4               [-1, 64, 50]          12,352
       BatchNorm1d-5               [-1, 64, 50]             128
         MaxPool1d-6               [-1, 64, 25]               0
            Conv1d-7              [-1, 128, 25]          24,704
       BatchNorm1d-8              [-1, 128, 25]             256
         MaxPool1d-9              [-1, 128, 12]               0
AdaptiveAvgPool1d-10               [-1, 128, 1]               0
           Linear-11                    [-1, 2]             258
================================================================
Total params: 63,490
Trainable params: 63,490
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.03
Forward/backward pass size (MB): 0.25
Params size (MB): 0.24
Estimated Total Size (MB): 0.52
```

Число параметров (n_mels=80, groups=1): **~63 490**

Обучение: PyTorch Lightning, 40 эпох, Adam lr=1e-3, ReduceLROnPlateau.

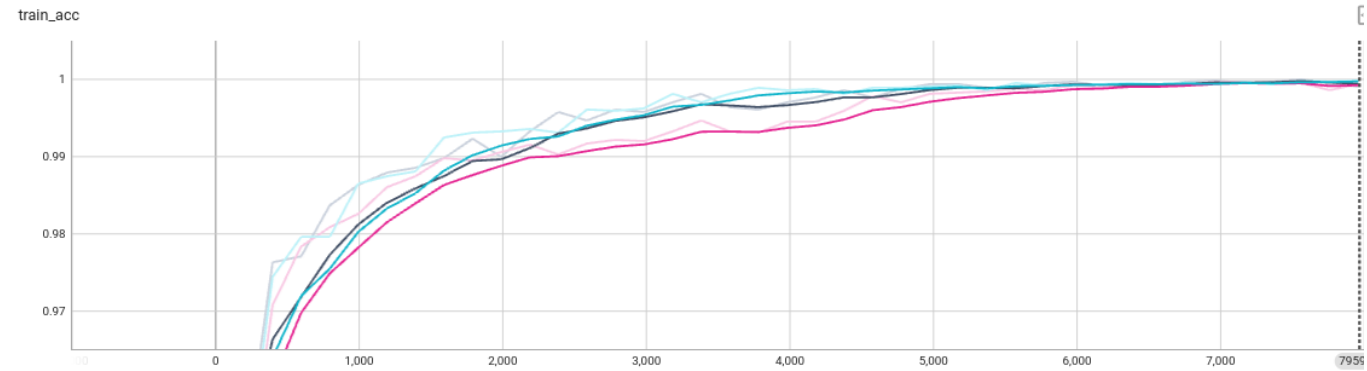
3. Эксперименты с n_mels [20, 40, 80]

Train loss:



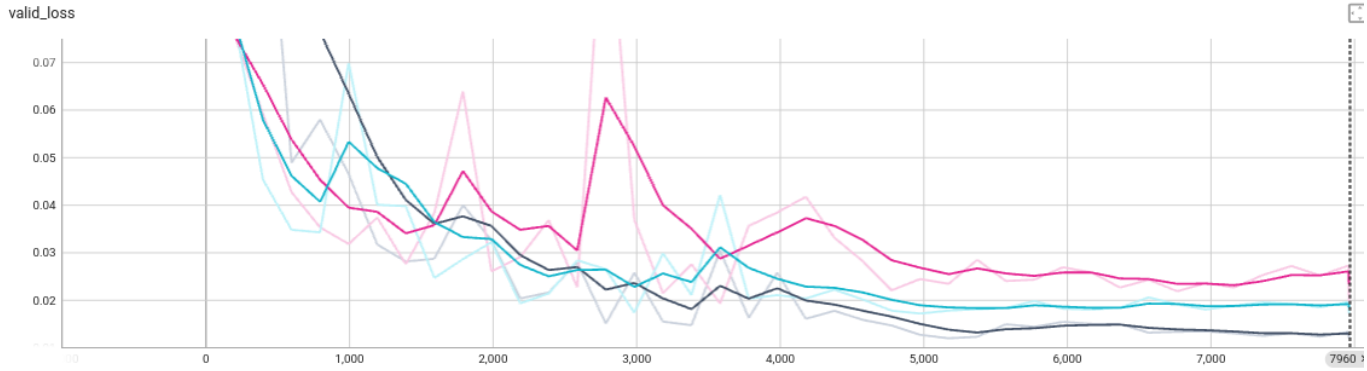
Run	Smoothed	Value	Step	Relative
mels_20_n_groups_1/version_0	0.0033	0.0029	7,959	9.313 min
mels_40_n_groups_1/version_0	0.0033	0.003	7,959	9.115 min
mels_80_n_groups_1/version_0	0.0033	0.0048	7,959	9.121 min

Train accuracy:



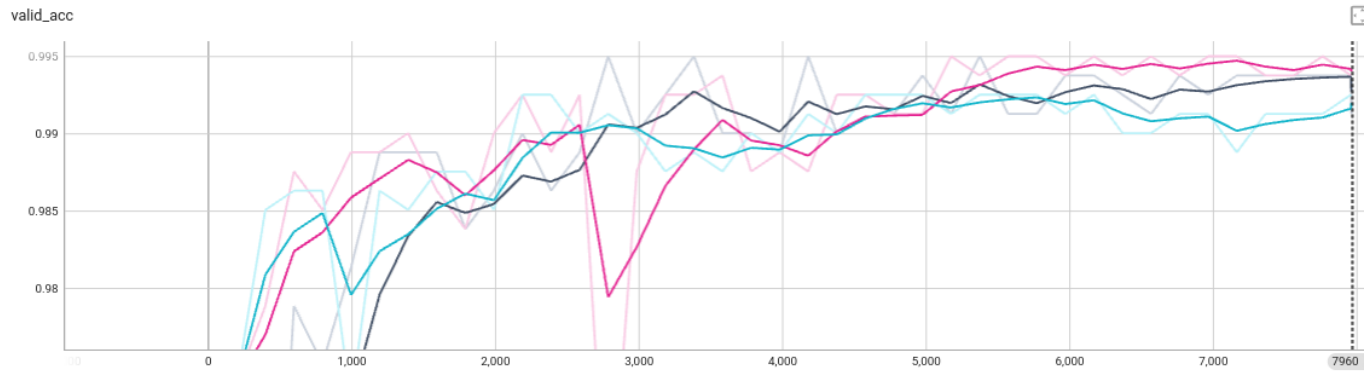
Run ↑	Smoothed	Value	Step	Relative
mels_20_n_groups_1/version_0	0.9992	0.9994	7,959	9.313 min
mels_40_n_groups_1/version_0	0.9997	0.9998	7,959	9.115 min
mels_80_n_groups_1/version_0	0.9994	0.9991	7,959	9.121 min

Validation loss:



Run ↑	Smoothed	Value	Step	Relative
mels_20_n_groups_1/version_0	0.0233	0.0192	7,960	9.366 min
mels_40_n_groups_1/version_0	0.0183	0.0171	7,960	9.167 min
mels_80_n_groups_1/version_0	0.0125	0.0119	7,960	9.172 min

Validation accuracy:



Run ↑	Smoothed	Value	Step	Relative
mels_20_n_groups_1/version_0	0.994	0.9938	7,960	9.366 min
mels_40_n_groups_1/version_0	0.992	0.9925	7,960	9.167 min
mels_80_n_groups_1/version_0	0.9927	0.9913	7,960	9.172 min

Тестовые метрики (n_mels=80):

Test metric	DataLoader 0
test_acc	0.9951456189155579
test_loss	0.012745670042932034

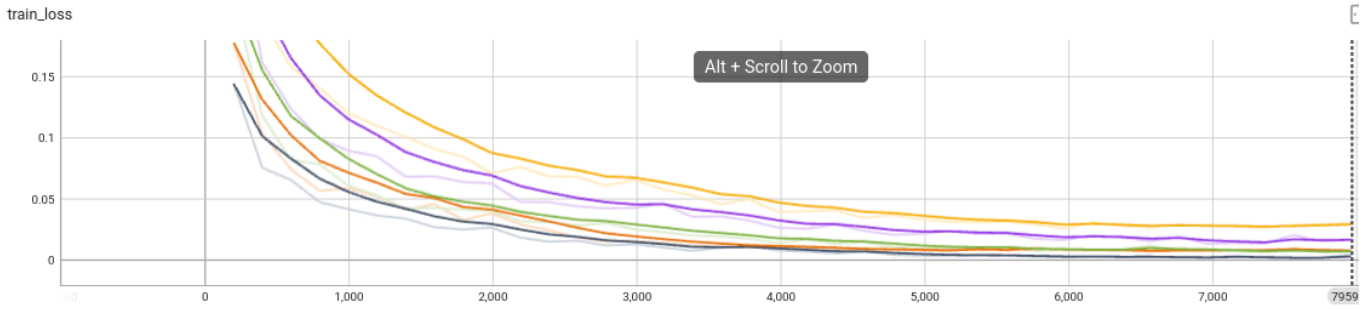
Модель с n_mels=20 показала лучшую valid_acc, хотя train loss немного выше, больше фильтров даёт чуть хуже обобщение на текущей задаче ввозможно это связано с какими-то выбросами в определенном спектре сигнала (шум), которые сглаживаются при усреднении на меньшем количестве бинов. Хотя различия не значительные между результатами и возможно в целом для такой простой задачи классификации количество фильтров больше 20 просто избыточно.

Итоговая точность на тестовой подвыборке (n_mels=80): **99.51%**

---

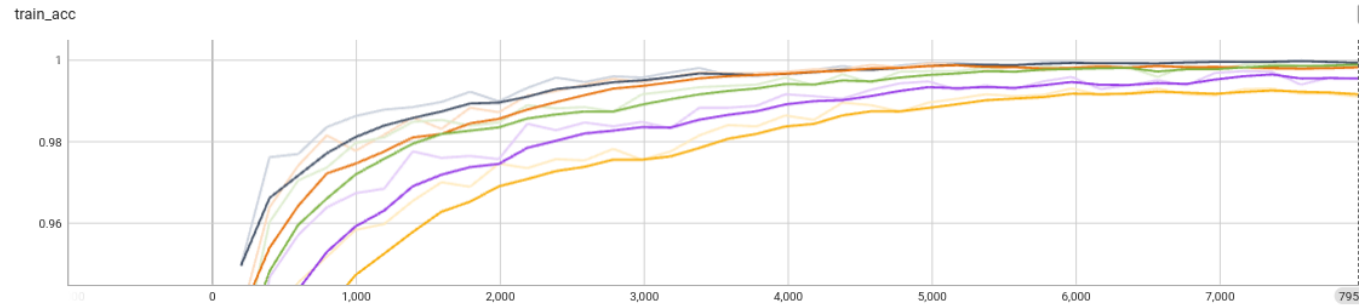
4. Эксперименты с параметром groups в Conv1d

Train loss:



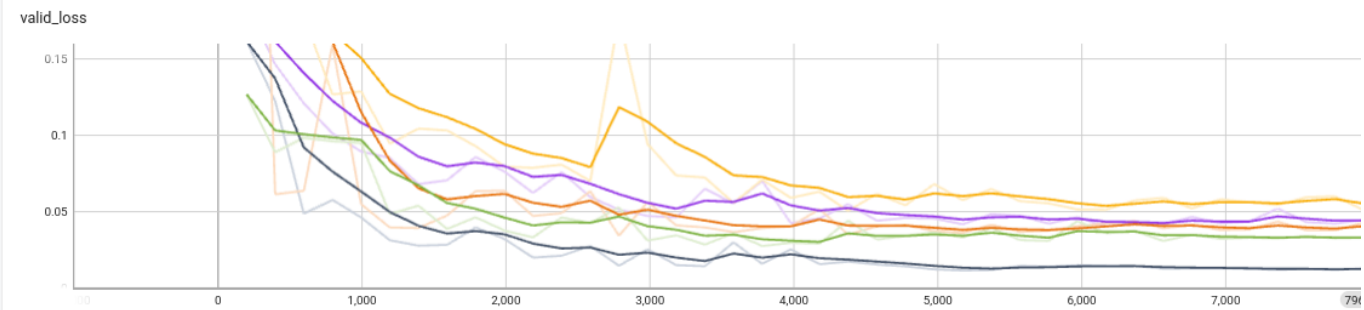
Run	Smoothed	Value	Step	Relative
mels_80_n_groups_1/version_0	0.0033	0.0048	7,959	9.121 min
mels_80_n_groups_16/version_0	0.0295	0.0305	7,959	9.161 min
mels_80_n_groups_2/version_0	0.0082	0.0074	7,959	9.061 min
mels_80_n_groups_4/version_0	0.0075	0.0073	7,959	9.013 min
mels_80_n_groups_8/version_0	0.0169	0.0173	7,959	9.118 min

Train accuracy:



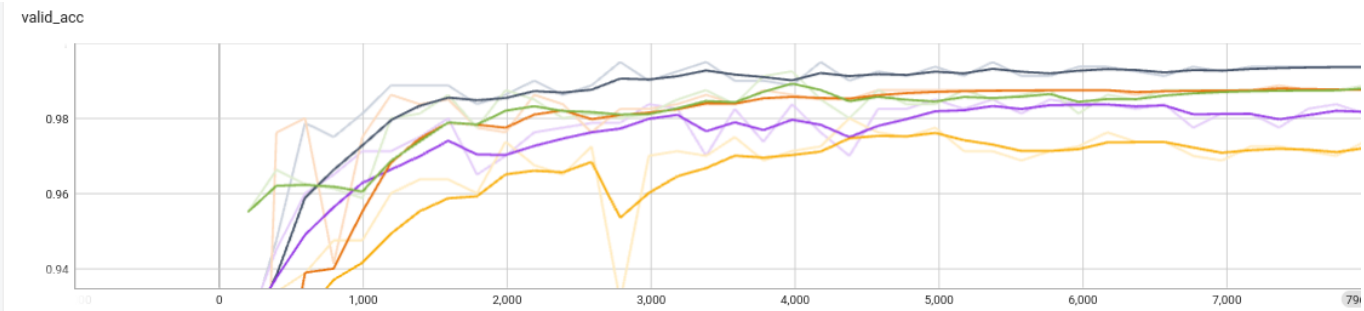
Run	Smoothed	Value	Step	Relative
mels_80_n_groups_1/version_0	0.9994	0.9991	7,959	9.121 min
mels_80_n_groups_16/version_0	0.9917	0.991	7,959	9.161 min
mels_80_n_groups_2/version_0	0.9982	0.9986	7,959	9.061 min
mels_80_n_groups_4/version_0	0.9989	0.9994	7,959	9.013 min
mels_80_n_groups_8/version_0	0.9955	0.9954	7,959	9.118 min

Validation loss:



Run	Smoothed	Value	Step	Relative
mels_80_n_groups_1/version_0	0.0125	0.0119	7,960	9.172 min
mels_80_n_groups_16/version_0	0.0536	0.0508	7,960	9.212 min
mels_80_n_groups_2/version_0	0.0383	0.0345	7,960	9.113 min
mels_80_n_groups_4/version_0	0.0311	0.0278	7,960	9.064 min
mels_80_n_groups_8/version_0	0.043	0.0407	7,960	9.169 min

Validation accuracy:



Run	Smoothed	Value	Step	Relative
mels_80_n_groups_1/version_0	0.9927	0.9913	7,960	9.172 min
mels_80_n_groups_16/version_0	0.9753	0.9801	7,960	9.212 min
mels_80_n_groups_2/version_0	0.9856	0.9826	7,960	9.113 min
mels_80_n_groups_4/version_0	0.9893	0.9913	7,960	9.064 min
mels_80_n_groups_8/version_0	0.9826	0.9838	7,960	9.169 min

Время эпох и параметры модели:

```
groups=1 | params=63,490 | flops=3,867,776 | time=12.03s
groups=2 | params=32,258 | flops=1,960,576 | time=12.07s
groups=4 | params=16,642 | flops=1,006,976 | time=12.06s
groups=8 | params=8,834 | flops=530,176 | time=12.43s
groups=16 | params=4,930 | flops=291,776 | time=12.26s
```

На основе проведённых экспериментов можно сделать вывод, что увеличение числа групп ухудшает качество модели: при groups=1 получаем val accuracy 0.993, а при groups=16 - 0.975. При этом количество параметров падает примерно в 13 раз (с 63 490 до 4 930), FLOPs-также примерно в 13 раз (с 3 867 776 до 291 776). Время на 1 эпоху остаётся практически неизменным при разных значениях groups - это объясняется тем, что модель достаточно мала, и основные затраты времени приходятся на подготовку и загрузку данных на GPU. Вычисления на модели размером 8-63к параметров занимают незначительное время по сравнению с IO.